

CMPE679: Deep Learning
Lab 2
English-to-German Neural Machine Translation Using an Encoder–
Decoder Transformer

Gloria Mbaka
Performed: 2/10/2026
Submitted: 2/12/2026

Instructor: Dr. Liu, Dongfang
TA: Ishan Renjen

By submitting this report, I attest that I neither have given nor have received any unauthorized assistance (including writing, collecting data, plotting figures, tables, or graphs, or using previous student reports as a reference), and I further acknowledge that giving or receiving such assistance will result in a failing grade for this course.

Your Signature: CGM

1. Abstract

This laboratory implemented an encoder–decoder transformer for English-to-German neural machine translation using the WMT14 parallel corpus. The model followed the core design described in Attention Is All You Need, including sinusoidal positional encoding, multi-head self-attention, cross-attention, residual connections, layer normalization, and position-wise feedforward networks. A byte-pair encoding tokenizer was trained on the English and German text, and the tokenized dataset was cached to support restartable training. Because the dataset contains 4,508,785 training pairs, training was organized with checkpointing, chunked loading, and saved metrics so that long runs could resume after interruptions. At the time of this final report update, four full epochs had been completed. Losses improved steadily across all completed epochs: training loss decreased from 3.3574 to 2.5727, validation loss decreased from 3.3906 to 2.6733, and test loss decreased from 3.5169 to 2.7165. Translation results show that the model can already produce several meaningful English-to-German outputs, especially on shorter inputs, although longer news-style sentences still remain unstable. A random-subset BLEU evaluation also remained low, which confirms that sequence-level translation quality is still limited despite good token-level loss improvement.

2. Introduction

Machine translation is a fundamental task in natural language processing that involves converting text from one language into another while preserving meaning and grammatical structure. Traditional machine translation systems relied heavily on statistical methods and phrase-based models, which required significant feature engineering and struggled with long-range contextual dependencies. Deep learning approaches have significantly improved translation performance by enabling models to learn complex linguistic patterns directly from data. Early neural machine translation systems used recurrent neural networks with attention mechanisms to model sequential dependencies between source and target languages. However, recurrent architectures suffer from limited parallelization and difficulty capturing long-range dependencies.

The transformer architecture addresses these limitations by eliminating recurrence entirely and relying solely on attention mechanisms to model relationships between tokens. In an encoder–decoder transformer, the encoder processes the input sequence and produces contextualized representations, while the decoder generates the output sequence by attending to both previously generated tokens and the encoder outputs. The objective of this laboratory is to implement a full encoder–decoder transformer model for English-to-German translation and train it using a large parallel corpus. The implementation requires constructing multi-head attention layers, positional encodings, encoder and decoder blocks, and the final projection layer used to generate vocabulary probabilities. The experiment provides practical experience with large-scale sequence modeling and highlights the computational demands associated with training modern deep learning architectures.

3. Dataset Description

The dataset used for this laboratory is the WMT14 English–German translation dataset. This dataset contains millions of sentence pairs in which each English sentence corresponds to a German translation. The dataset is commonly used as a benchmark for evaluating machine translation systems.

Each training example consists of a parallel pair

$$x = (x_1, x_2, \dots, x_n)$$

representing the English source sentence and

$$y = (y_1, y_2, \dots, y_m)$$

representing the corresponding German translation.

The dataset is divided into three subsets; training set used for learning model parameters, validation set used to monitor training progress and detect overfitting and test set used to evaluate final translation performance. Before training, sentences were tokenized using a Byte Pair Encoding (BPE) tokenizer. BPE constructs a subword vocabulary by iteratively merging frequently occurring character sequences. This allows rare words to be represented as combinations of smaller subword units while maintaining a manageable vocabulary size.

Special tokens were added to support sequence modeling:

PAD – used to pad sequences to equal length

UNK – represents unknown tokens

BOS – indicates the beginning of a sequence

EOS – indicates the end of a sequence

After tokenization, sentences were truncated or padded to a maximum sequence length to ensure consistent tensor dimensions for batch processing.

4. Tokenization and Preprocessing

Text preprocessing converts raw sentences into numerical representations suitable for neural network input. The tokenizer was trained using the training portion of the dataset and generates a vocabulary consisting of subword units.

Each input sentence is converted into a sequence of integer token identifiers

$$x \in \mathbb{N}^T$$

where T denotes the maximum sequence length.

Similarly, target sentences are encoded as token sequences that the decoder learns to generate during training. During training, the target sequence is shifted by one position to allow the decoder to predict the next token based on previously generated tokens. This process is known as teacher forcing and enables efficient training of sequence generation models. Padding tokens are used to ensure uniform sequence lengths across batches, and masking mechanisms are applied to prevent attention operations from incorporating padded positions.

5. Transformer Model Architecture

The implemented model follows the encoder–decoder transformer architecture consisting of stacked attention and feedforward layers.

5.1 Positional Encoding

Since the transformer does not contain recurrence or convolution, positional encoding is used to provide information about token order. The positional encoding is added to the input embeddings and is defined using sinusoidal functions:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

These encodings allow the model to infer relative and absolute token positions.

5.2 Multi-Head Attention

Attention mechanisms allow the model to determine which tokens are most relevant when generating representations.

Given query (Q), key (K), and value (V) matrices, scaled dot-product attention is computed as

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-head attention performs this computation in parallel across multiple attention heads. Each head learns different relationships between tokens.

5.3 Encoder

The encoder consists of six identical layers. Each layer contains:

1. Multi-head self-attention
2. Position-wise feedforward network
3. Residual connections
4. Layer normalization

Self-attention allows each token in the source sentence to attend to all other tokens in the sentence.

5.4 Decoder

The decoder also consists of six stacked layers. Each decoder layer contains:

1. Masked self-attention
2. Cross-attention with encoder outputs
3. Feedforward network
4. Residual connections and layer normalization

Masked self-attention prevents the decoder from attending to future tokens during training.

Cross-attention allows the decoder to align generated tokens with relevant positions in the input sequence.

6. Training Procedure

Training was performed using the Adam optimizer with the inverse square-root learning-rate schedule described in the transformer paper. The model parameters were updated by minimizing token-level cross-entropy loss between predicted target tokens and reference target tokens. Training used cached tokenized data, gradient accumulation, checkpointing, and chunked loading of the training set in order to handle the size of the WMT14 corpus within the available Colab GPU memory. Latest and best checkpoints were saved during training, and metrics were written to JSON and CSV files after each completed epoch so that progress could be recovered after interruptions.

The loss function for sequence prediction is defined as

$$L = - \sum \log P(y_t | y_1, \dots, y_{t-1}, x)$$

where x represents the source sentence and y represents the target sentence.

Checkpointing was implemented during training to allow the model to resume training after interruptions. This is necessary due to the large dataset size and long training times.

Training was performed using GPU acceleration through Google Colab. Because the dataset contains millions of sentence pairs, training progress is computationally intensive and requires extended training time to reach convergence.

7. Results and Analysis

Training was carried out on the WMT14 English–German dataset using Google Colab with checkpointing enabled. The final pipeline used cached tokenized splits together with chunked training so that the model could continue after runtime interruptions. By the time of this report, four full epochs had been completed successfully, and the saved checkpoints, metrics files, plots, and qualitative translations were all generated from the trained model.

The recorded losses show a clear and consistent downward trend across epochs. This is the strongest quantitative evidence that the implementation is working correctly and that the model is learning useful cross-lingual structure. The train, validation, and test losses remain close to one another, which suggests that the model is generalizing reasonably well and is not showing strong overfitting at this stage.

Table 1 summarizes the completed epoch metrics used in the discussion below.

Table 1: Epoch Metrics

Epoch	Train Loss	Validation Loss	Test Loss
1	3.3574	3.3906	3.5169
2	2.9546	2.9956	3.0751
3	2.7270	2.7172	2.7687
4	2.5727	2.6733	2.7165

A separate epoch-level loss plot was generated from the saved checkpoint history (Figure 1). All three losses decrease steadily, and improvement slows slightly after epoch 3, which suggests that the model is approaching convergence under the current settings rather than diverging or memorizing the training set.

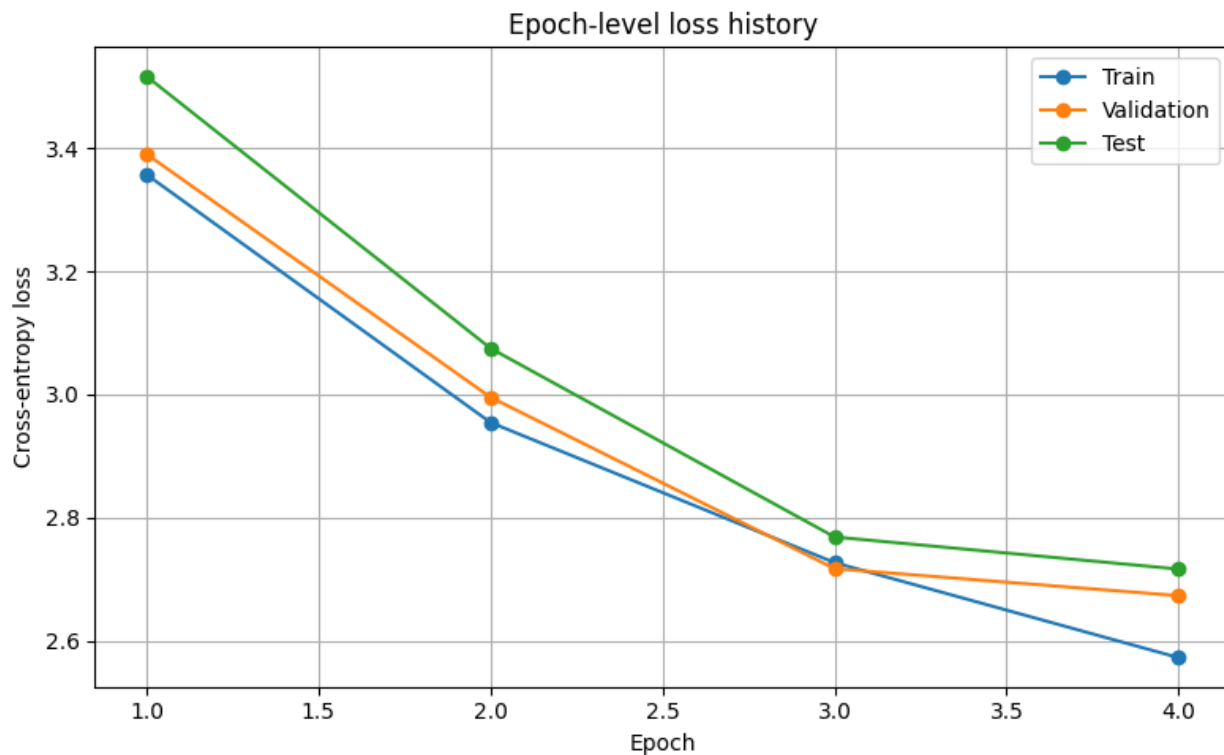


Figure 1 Epoch vs train/validation/test loss

Qualitative evaluation was carried out by translating English sentences into German with beam-search decoding and then translating the model output back into English with Google Translate for semantic checking. Because the decoder still produced many empty outputs on difficult examples, a filtered sampling strategy was implemented so that only completed model translations were reported. Ten usable dataset translations were collected after 168 attempts. This result is important because it shows both progress and limitation: the model has learned enough to produce meaningful outputs on selected examples, but broad robustness on random test sentences is still weak.

A random BLEU evaluation was also performed on 200 sampled test examples using faster greedy decoding. Only 8 of those 200 examples produced non-empty predictions, and the BLEU score on the valid predictions was 2.13. This low BLEU score is consistent with the qualitative observations. It shows that the model has not yet become a strong general translator, even though the loss curves improve steadily. In other words, token-level optimization is improving, but sequence-level generation quality still needs more training and better decoding behavior.

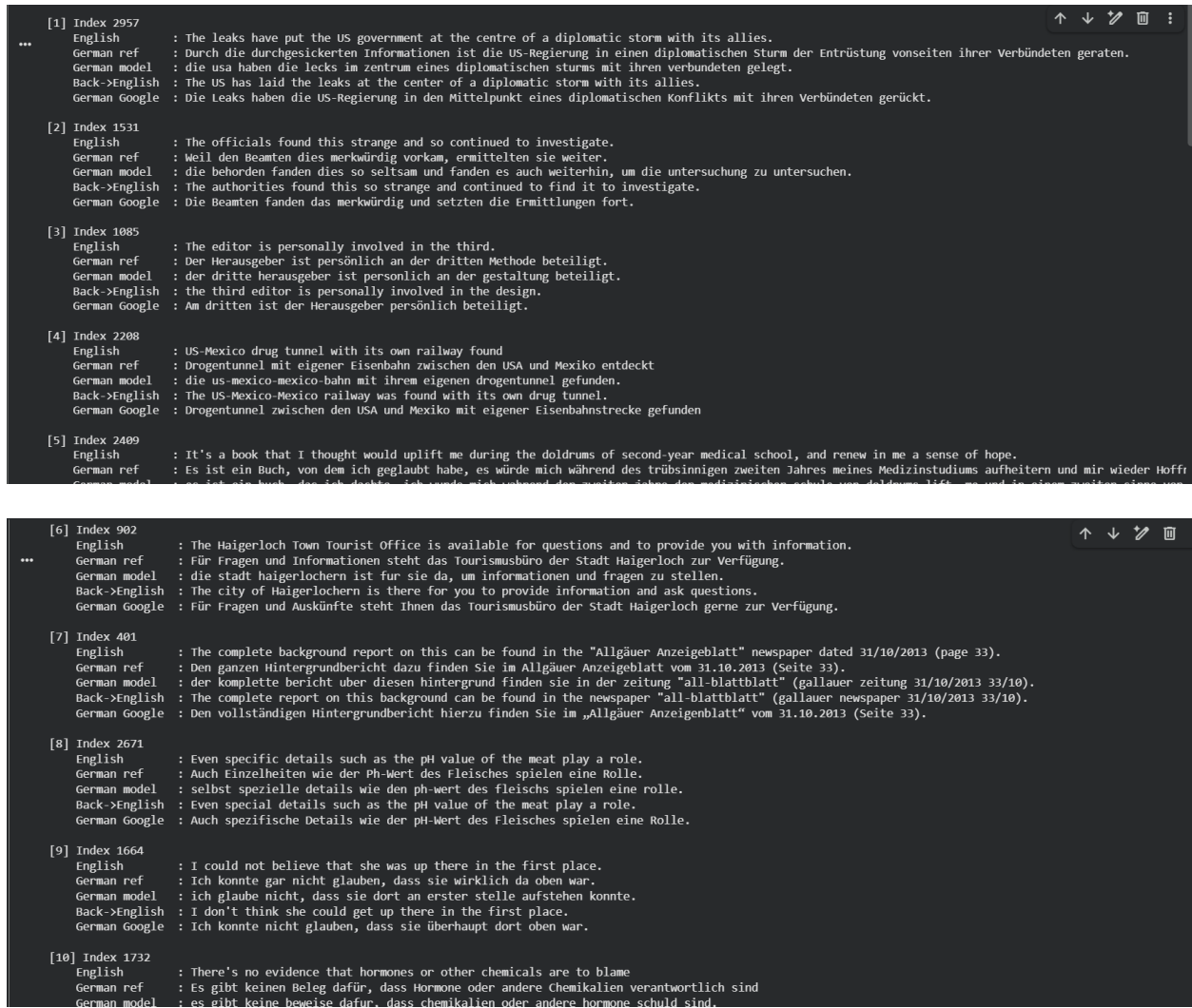


Figure 2: Screenshot of 10 notebook translations with back-translation

8. Issues Encountered

Several practical issues were encountered during this laboratory. The first was the size of the WMT14 dataset, which made tokenization, caching, and epoch-level training time substantial. The second was GPU memory and runtime limitations in Colab, which required chunked training, checkpointing, and careful control of background processes. A third issue involved decoder instability and occasional non-finite losses when training with automatic mixed precision at aggressive learning rates, so later runs were stabilized by disabling AMP. Another issue was that many random sentences initially decoded to empty outputs because the model favored early EOS termination. To address this, the notebook was updated to use better decoding settings, filtered translation sampling, checkpoint-aware evaluation, saved metrics, and Google-

assisted back-translation for qualitative analysis. These script changes were important in moving from a partially working implementation to a reproducible training and evaluation pipeline.

9. Conclusion

This laboratory successfully implemented a complete encoder–decoder transformer for English-to-German translation using the WMT14 dataset. The final system included tokenizer training, cached preprocessing, chunked checkpointed training, epoch-level metric saving, saved model artifacts, qualitative translation generation, and BLEU evaluation. Across four completed epochs, training, validation, and test losses all decreased steadily, which confirms that the model architecture and optimization pipeline were functioning correctly.

At the same time, the translation results show that the model is not yet fully mature. It can produce several meaningful translations, especially on shorter or simpler inputs, but it still struggles on many harder news-domain sentences. The filtered qualitative examples, back-translation analysis, and low BLEU score all point to the same conclusion: the model is learning real English-to-German structure, but additional training and decoding refinement are still needed for reliable sequence-level translation quality. Future improvements should include more epochs, stronger decoding control, additional regularization experiments, and possibly a larger or better-tuned model. Overall, the lab objectives were met: the transformer was implemented, trained, evaluated, and used to produce the required translation outputs.

11. Questions

Q1: Feedback on the Lab

This laboratory provided valuable experience implementing the transformer architecture and understanding attention-based sequence modeling. However, training large transformer models on very large datasets can be computationally demanding when limited GPU resources are available. Providing a smaller subset of the dataset for experimentation could make the lab more manageable while still demonstrating the core concepts.

Q2: Explain Cross-Attention

Cross-attention allows the decoder to attend to the encoder outputs while generating each token in the target sequence. In this mechanism, the queries originate from the decoder hidden states, while the keys and values come from the encoder outputs. This allows the decoder to dynamically focus on relevant portions of the input sentence when predicting each output token. For example, when generating a particular German word, the decoder can attend to the corresponding English word or phrase in the encoder representation. Cross-attention is essential in translation because it enables alignment between source and target tokens.

Q3: Difference Between Encoder–Decoder Transformers and Decoder-Only Transformers

Encoder–decoder transformers process input and output sequences separately. The encoder processes the entire input sequence and produces contextualized representations. The decoder then generates the output sequence while attending to both previous outputs and encoder representations. Decoder-only transformers generate text autoregressively using only previously generated tokens. These architectures are commonly used for language modeling tasks such as text generation. Encoder–decoder models are particularly suitable for tasks that require transforming one sequence into another, such as translation or summarization.

References

- [1] A. Vaswani et al., “Attention Is All You Need,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [2] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in International Conference on Learning Representations (ICLR), 2015.
- [3] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A Method for Automatic Evaluation of Machine Translation,” in Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002.
- [4] P. Koehn, “Europarl: A Parallel Corpus for Statistical Machine Translation,” in MT Summit, 2005.
- [5] M. Ott et al., “Scaling Neural Machine Translation,” in Proceedings of the Third Conference on Machine Translation (WMT), 2018.
- [6] Hugging Face, “Datasets and Tokenizers Documentation,” 2024. [Online]. Available: <https://huggingface.co/docs>
- [7] PyTorch Team, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in NeurIPS, 2019.
- [8] WMT14 Dataset, “ACL 2014 Workshop on Statistical Machine Translation,” [Online]. Available: <http://www.statmt.org/wmt14/>
- [9] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in ACL, 2016.
- [10] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in EMNLP, 2020.